



No SQL





NOSQL

- “not only SQL.”
- NoSQL databases are databases that store data in a format other than **relational tables**.
- NoSQL databases or non-relational databases don't store **relationship data well**.
- NoSQL databases can store **relationship data**—they just store it differently than relational databases do.
- when compared with SQL databases, many find modeling relationship data in NoSQL databases to be *easier* than in SQL databases, because **related data doesn't have to be split between tables**.
- NoSQL data models allow related data to be **nested within a single data structure**.
-

- NoSQL databases ("**not only SQL**") are **non tabular**, and store data **differently** than relational tables.

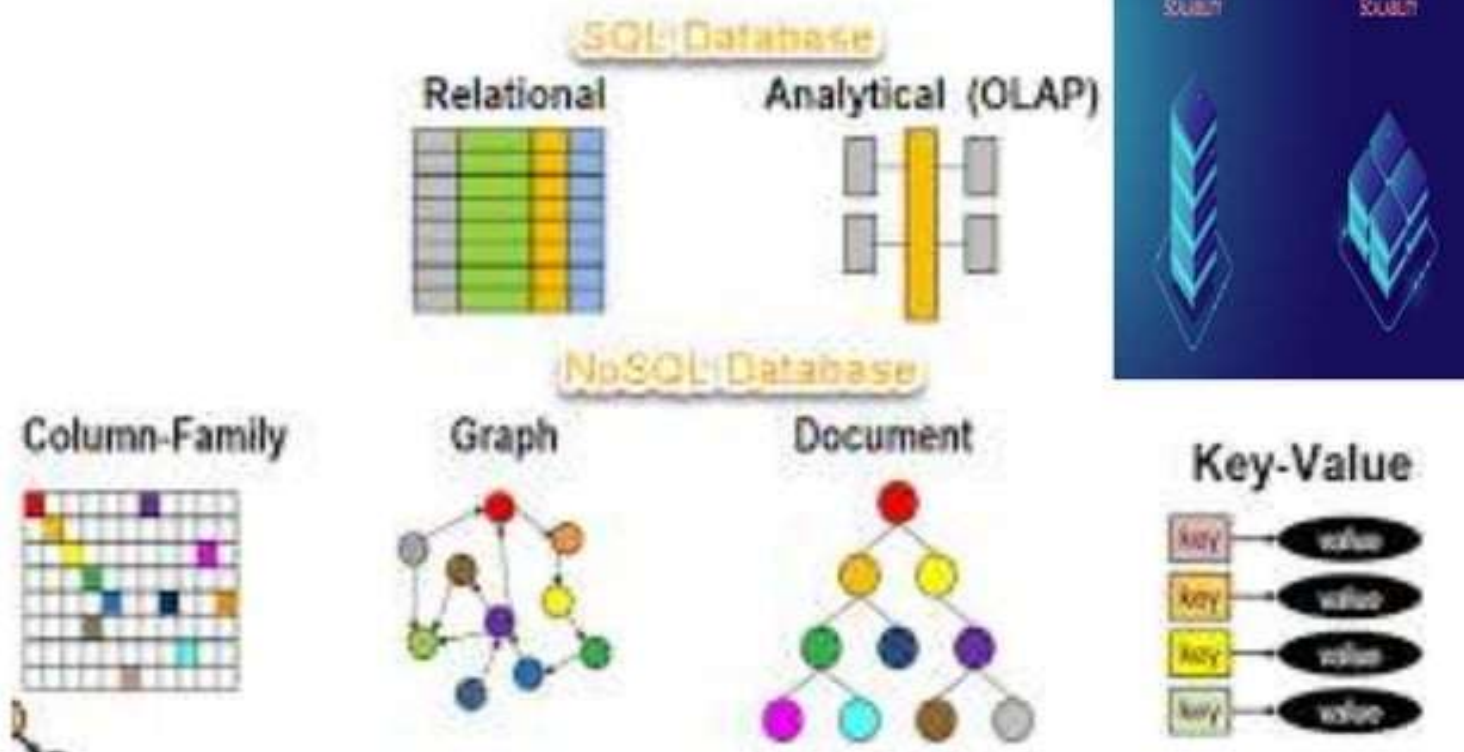
Nosql Data Model .



Document,
Key-value,
Wide column,
Graph

- They provide flexible **schemas** and scale easily with **large amounts of data and high user loads**.

Nosql Data Model .



Tools of NOSQL

EROSPIKE


cassandra

 Clustrix


Couchbase


amazon
DynamoDB

APACHE
HBASE

IBM
Cloudant

 MarkLogic

 memsql
Speed. Scale. Simplicity.

 mongoDB


NUODB

ORACLE
NOSQL
DATABASE

 riak

splice
MACHINE

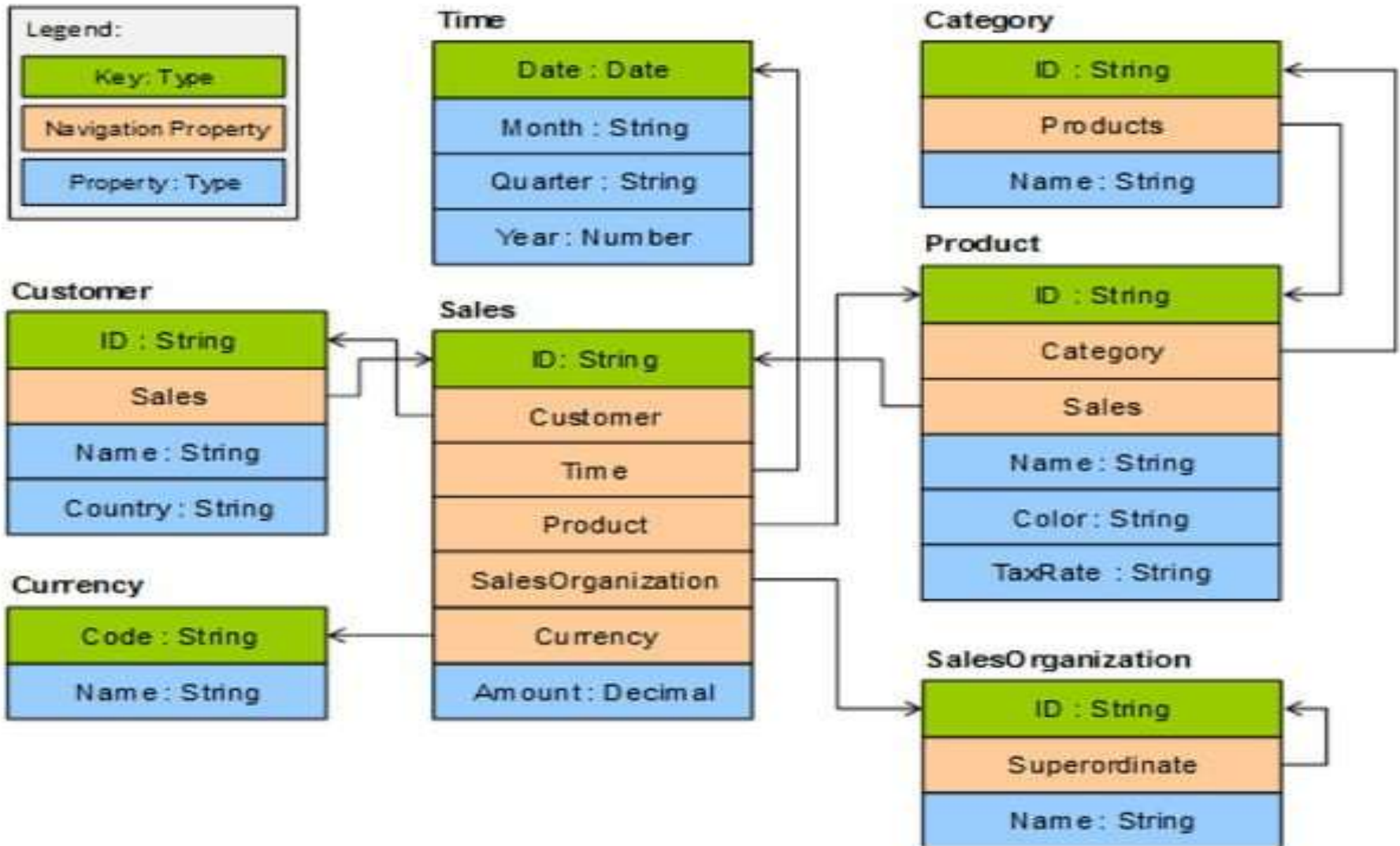
TRANS/ATTICE

 VOLTDB



Aggregates

- The relational model takes the information that we want to store and divides it into **tuples** (rows).
- **A tuple is a limited data structure:** It captures a set of values, so you **cannot** nest **one tuple** within another to get **nested records**, nor can you put a list of values or tuples within another.





Time

Date	Month	Quarter	Year
2012-01-01	2012-01	2012-1	2012
2012-04-01	2012-04	2012-2	2012
2012-04-10	2012-04	2012-2	2012
...			

Customers

ID	Name	Country
C1	Joe	USA
C2	Sue	USA
C3	Sue	Netherlands
C4	Luc	France

Sales

ID	Customer	Time	Product	Sales Organization	Currency	Amount
1	C1	2012-01-03	P3	US West	USD	1
2	C1	2012-04-10	P1	US West	USD	2
3	C1	2012-08-07	P2	US West	USD	4
4	C2	2012-01-03	P2	US East	USD	8
5	C2	2012-11-09	P3	US East	USD	4
6	C3	2012-04-01	P1	EMEA Central	EUR	2
7	C3	2012-08-06	P3	EMEA Central	EUR	1
8	C3	2012-11-22	P3	EMEA Central	EUR	2

Products

ID	Category	Name	Color	TaxRate
P1	PG1	Sugar	White	0.06
P2	PG1	Coffee	Brown	0.06
P3	PG2	Paper	White	0.14
P4	PG2	Pencil	Black	0.14

Categories

ID	Name
PG1	Food
PG2	Non-Food

SalesOrganizations

ID	Superordinate	Name
Sales		Corporate Sales
US	Sales	US
US West	US	US West
US East	US	US East
...		
EMEA	Sales	EMEA
EMEA Central	EMEA	EMEA Central
...		

Currencies

Code	Name
USD	US Dollar
EUR	Euro

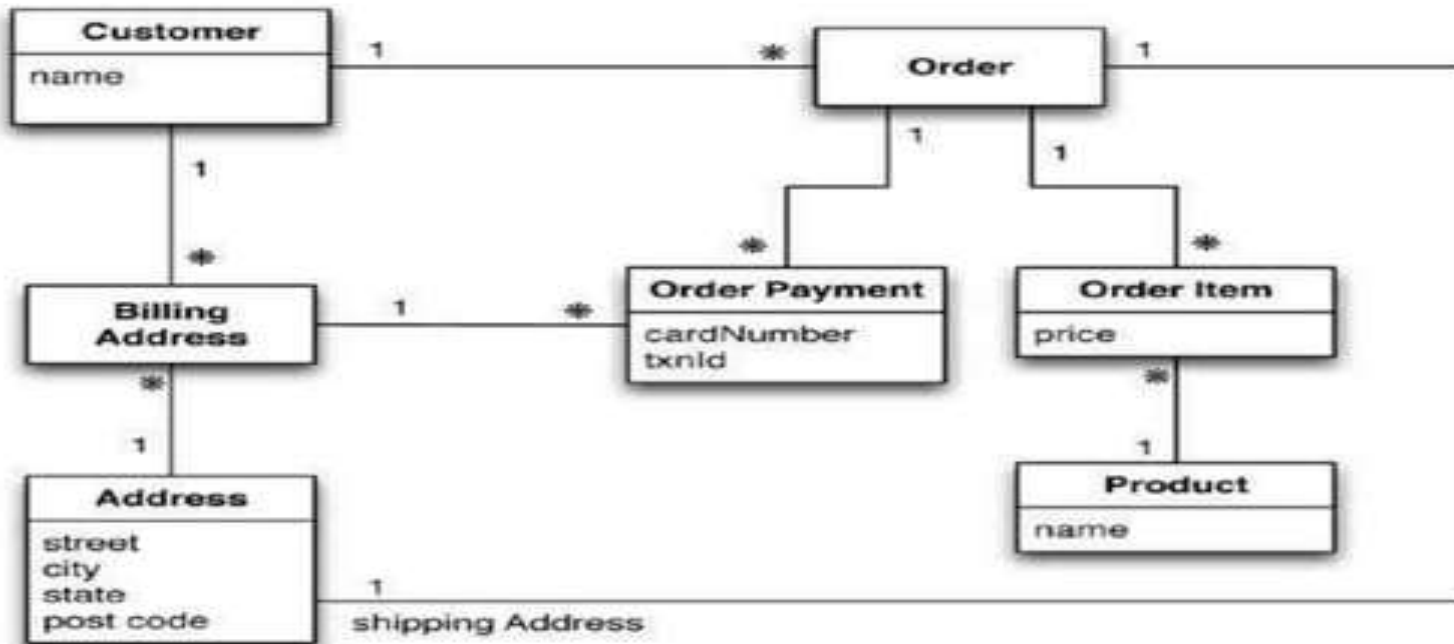


Aggregates

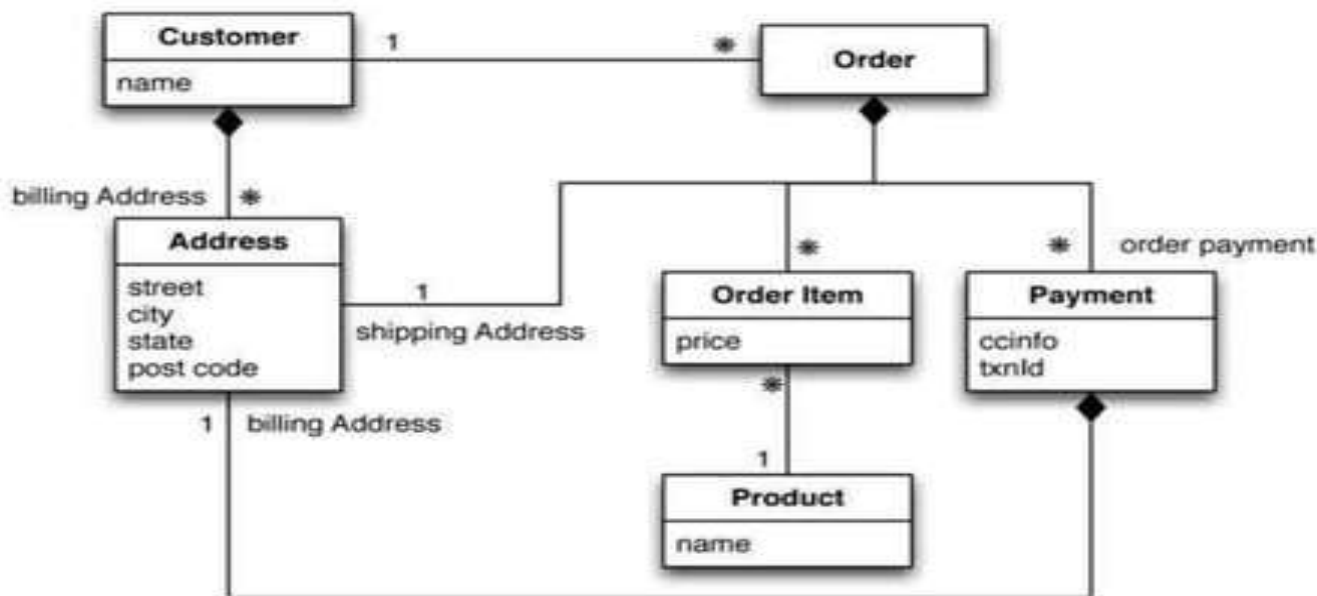
- db.orders.aggregate([
- { \$match: { status: "A" } },
- { \$group: { _id: "\$cust_id", total: { \$sum: "\$amount" } } }
-])



Aggregates relations



Aggregate data models





Key-Value and Document Data Models

- **key-value store**, we can only access an aggregate by lookup based on its key.
 - each item contains keys and values.
 - A value can typically only be retrieved by referencing its value, so learning how to query for a specific key-value pair is typically simple.
 - Key-value databases are great for use cases where you need to store large amounts of data but you **don't need to perform complex queries** to retrieve it.
- **Redis** and **DynanoDB** are popular key-value databases.
- **document database**, we can submit **queries** to the database based on the fields in the aggregate, we can retrieve **part** of the aggregate rather than the **whole** thing, and database can **create indexes based on the contents** of the aggregate.
 - Ex-JSON or XML structures.

Column-Family Stores

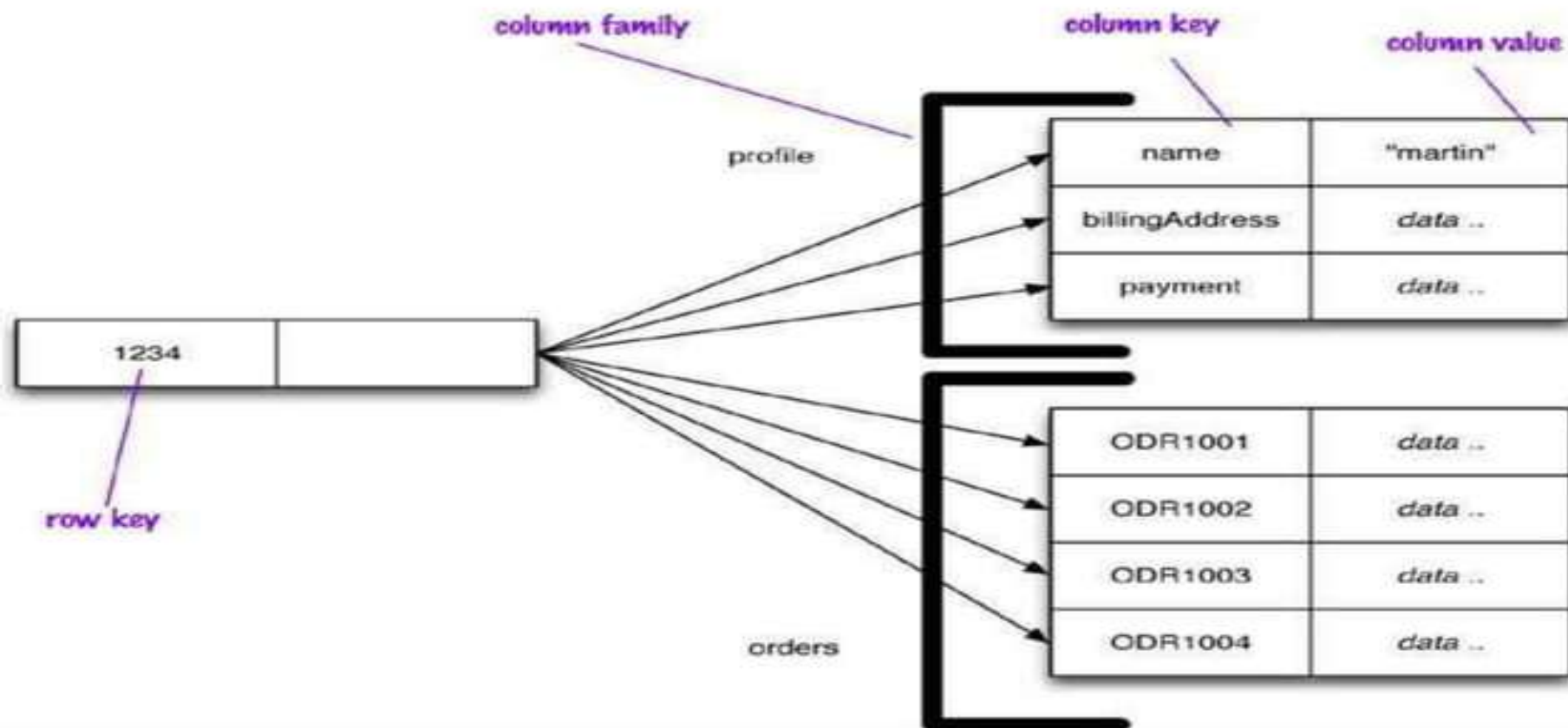
- NoSQL databases was **Google's BigTable**
- tabular structure which it realized with **sparse columns** and **no schema**
- Ex-**HBase** and **Cassandra**.
- **Pre-NoSQL column stores, such as C-Store** [\[C-Store\]](#)
- data in tables, rows, and dynamic columns.
- **Wide-column** stores provide a lot of flexibility over relational databases because each **row** is not required to have the same columns.
- Many consider wide-column stores to be **two-dimensional key-value** databases.
- Wide-column stores are great for when you need to **store large amounts of data** and you can predict what your query patterns will be.
- Wide-column stores are commonly used for storing **Internet of Things data** and user profile data.
- **Cassandra and HBase** are two of the most popular **wide-column** stores.

Column-Family Stores

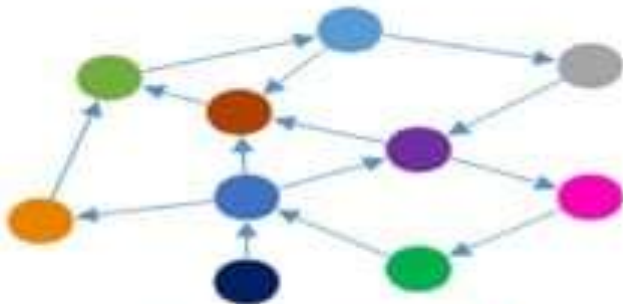
- **Row-oriented:** Each row is an aggregate with column families representing useful **chunks of** data (profile, order history) within that aggregate. (for example, customer with the ID of 1234)
- **Column-oriented:** Each column family defines a **record type** (e.g., customer profiles) with **rows for** each of the records. You then think of a row as the **join of records** in all column families.
- Cassandra uses the terms **“wide”** and **“skinny.”** **Skinny rows** have **few columns with the same columns used** across the many different rows.
- In this case, the column family defines a record type, each row is a record, and each column is a field.
- A **wide row has many columns** (perhaps thousands), with rows having very different columns.
- A wide column family models a list, with **each column being one element in that list.**



Representing customer information in a column-family structure



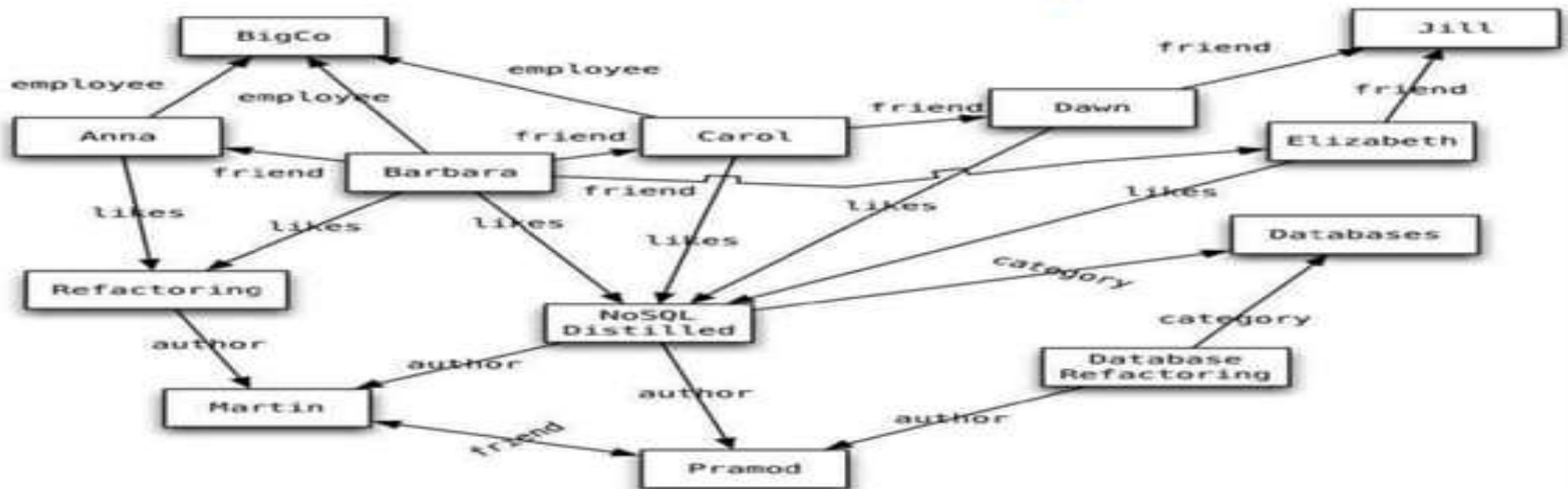
Graph Database



- **store data in nodes and edges.** 
- **Nodes** typically store information about **people, places, and things** while **edges** store information about the **relationships between the nodes**.
- Graph databases excel in use cases where you need to **traverse relationships** to look for patterns such as social networks, fraud detection, and recommendation engines.
- **Neo4j** and **JanusGraph** are examples of graph databases.

Graph Databases refer to a graph data structure of nodes connected by edges.

- aggregate-oriented data models of large records **with simple connections.**



RELATIONSHIPS

- relationship between a customer and all of his orders.
- many databases—even key-value stores—provide ways to make these relationships visible to the database.
- Document stores make the content of the aggregate available to the database to form indexes and queries.
- Relationships are always depends on the type of aggregate, it may be single or multiple aggregates.

- FlockDB is simply nodes and edges with no mechanism for additional attributes;
- Neo4J allows you to attach Java objects as properties to nodes and edges in a schemaless fashion
- Infinite Graph stores your Java objects, which are subclasses of its built-in types, as nodes and edges.



Schemaless Databases

schema less means the database **don't have fixed data structure**, such as **MongoDB**, it has JSON-style data store, you can change the data structure as you wish

//pseudo code

```
foreach (Record r in records) {  
  foreach (Field f in r.fields) {  
    print (f.name, f.value)  
  }  
}
```



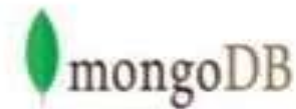
Faster process



Open Source



Sharding



Schemaless



Document based



No SQL Injection



Advantages of schemaless:

1. **Speed** for whole document requests
2. **Ability to store any format or data** - including documents with missing fields
3. Most technologies (e.g. Cassandra, Hadoop, Mondo) allow for **rapid and easy scaling** of servers (sharding/ clustering).
4. Some technologies allow for **indexing** - but at that point you are not really schemaless so you can have a nearly schemaless design with **one primary key** (say a document id) and required fields (like a timestamp) ... and still allow nearly anything else to be loaded in.
5. Great, solution **for collecting logs** (See Splunk)
6. A developer can build their own objects (schema) easily and change them on the fly (think Agile) without engaging a DBA.

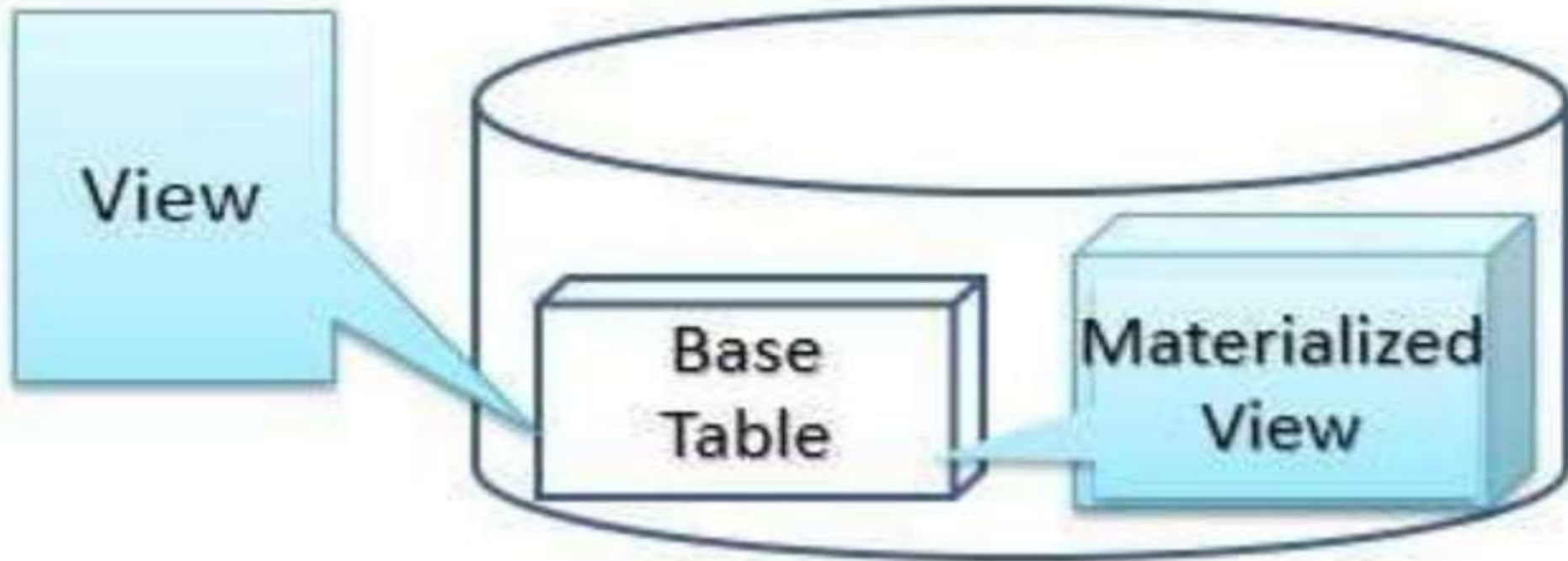


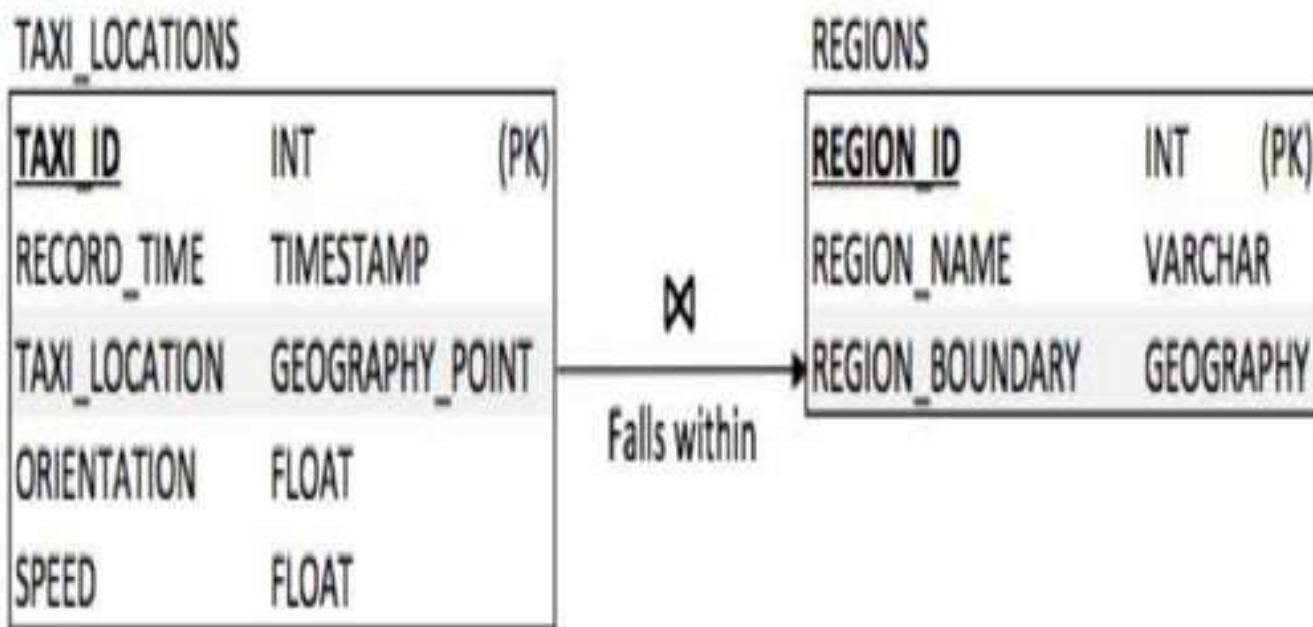
Materialized Views

- A view is like a relational table (it is a relation) but it's defined by **computation over the base tables**. When you access a view, the database computes the data in the view—a handy form of encapsulation.
- Views provide a mechanism to hide from the client whether data is **derived data or base data**—but can't avoid the fact that some views are expensive to compute.
- Aggregate-oriented databases often compute materialized views to provide **data organized** differently from their primary aggregates. This is often done with map-reduce computations.
- note:
- Aggregate-oriented databases make inter-aggregate relationships more difficult to handle than intra-aggregate relationships.
- Graph databases organize data into node and edge graphs; they work best for data that has complex relationship structures.
- Schemaless databases allow you to freely add fields to records, but there is usually an implicit schema expected by users of the data.



Materialized Views



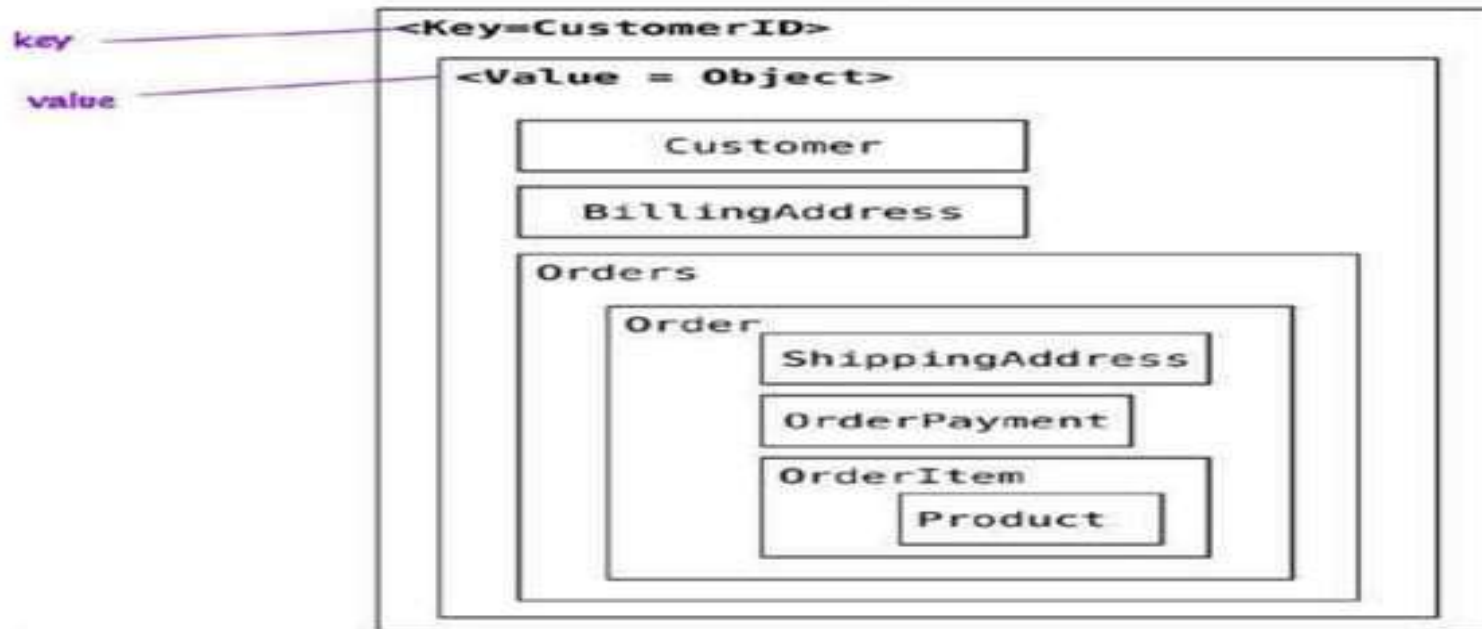




Basic	A View is never stored it is only displayed.	A Materialized View is stored on the disk.
Define	View is the virtual table formed from one or more base tables or views.	Materialized view is a physical copy of the base table.
Update	View is updated each time the virtual table (View) is used.	Materialized View has to be updated manually or using triggers.
Speed	Slow processing.	Fast processing.
Memory usage	View do not require memory space.	Materialized View utilizes memory space.
Syntax	Create View V As	Create Materialized View V Build [clause] Refresh [clause] On [Trigger]

Modeling for Data Access

- how the data is going to be read as well as what are the side effects on data related to those aggregates.
- data for the customer is embedded using a **key-value store**





Distribution Models

- The primary driver of interest in NoSQL has been its ability to run databases on a **large cluster**.
- As **data volumes increase**, it becomes more **difficult and expensive** to scale up—buy a bigger server to run the database on.
- A more appealing option is to scale **out—run the database on a cluster of servers**.
- Aggregate orientation fits well with scaling out because the aggregate is a natural unit **to use for distribution**.



Distribution Models

- there are two paths to data distribution:
- **REPLICATION**
- **SHARDING.**
- Replication takes the same data and copies it over multiple nodes.
- Sharding puts different data on different nodes. You can use either or both of them.
- Replication comes into two forms:
 - master-slave
 - peer-to-peer.
-



Parallel vs. Distributed DBMS

Parallel DBMS

- Parallelization of various operations
- e.g. loading data, building indexes, evaluating queries
- Data may or may not be distributed initially
- Distribution is governed by performance consideration

Distributed DBMS

- Data is physically stored across different sites
 - Each site is typically managed by an independent DBMS
- Location of data and autonomy of sites have an impact on Query opt., Conc. Control and recovery
- Also governed by other factors:
 - increased availability for system crash
 - local ownership and access



Two desired properties and recent trends

- Data is stored at several sites, each managed by a DBMS that can run independently
 1. **Distributed Data Independence**
 - Users should not have to know where data is located
 2. **Distributed Transaction Atomicity**
 - Users should be able to write transactions accessing multiple sites just like local transactions
 - These two properties are in general desirable, but not always efficiently achievable
 - e.g. when sites are connected by a slow long-distance network
 - Even sometimes not desirable for globally distributed sites
 - too much administrative overhead of making location of data transparent (not visible to the user)
 - Therefore not always supported
 - Users have to be aware of where data is located



Single Server

- Run the database on a **single machine** that handles all the reads and writes to the data store.
- data store is busy because **different people** are **accessing different parts** of the dataset. In these circumstances we can support **horizontal scalability** by putting different parts of the data onto different servers—a technique that's called **sharding**



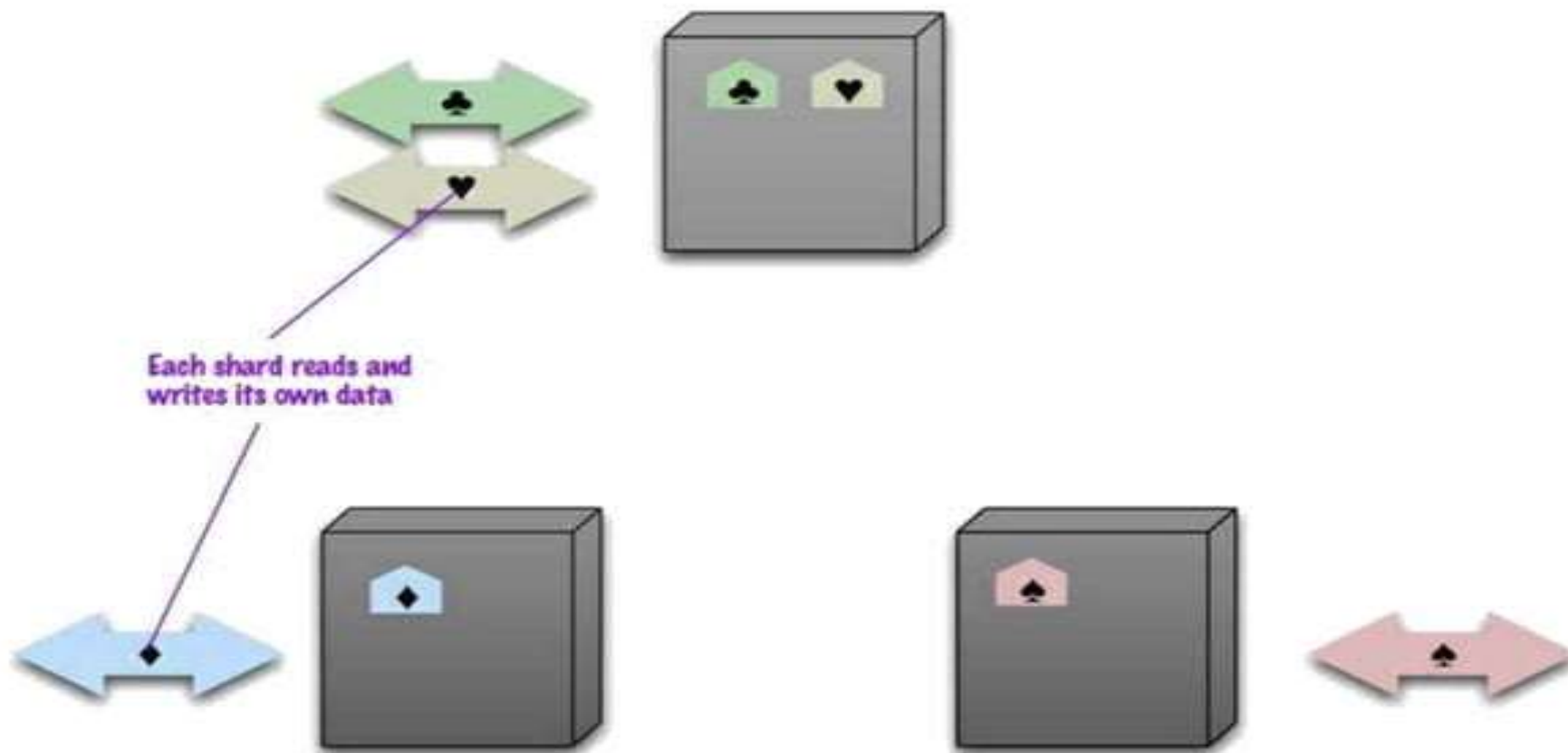
SHARDING

Replication = Create multiple copies of each database partition. Replication can be synchronous or asynchronous. Spread queries across these replicas. Goals: scalability and availability.

Sharding = horizontal partitioning by some key, and storing partitions on different servers. Data is denormalized to avoid cross-shard operations (no distributed joins). Split the shards as data volumes or access grows. Goals: massive scalability.

SHARDING

Sharding puts different data on separate nodes, each of which does its own reads and writes.





SHARDING

- You might put all customers with surnames starting from A to D on one shard and E to G on another.
- This complicates the programming model, as application code needs to ensure that queries are distributed across the various shards.
- Furthermore, rebalancing the sharding means changing the application code and migrating the data.
- Many NoSQL databases offer **auto-sharding**, where the database takes on the responsibility of allocating data to shards and ensuring that data access goes to the right shard.
- This can make it much easier to use sharding in an application.

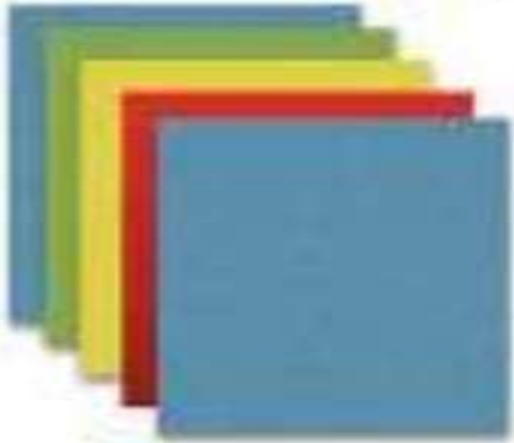


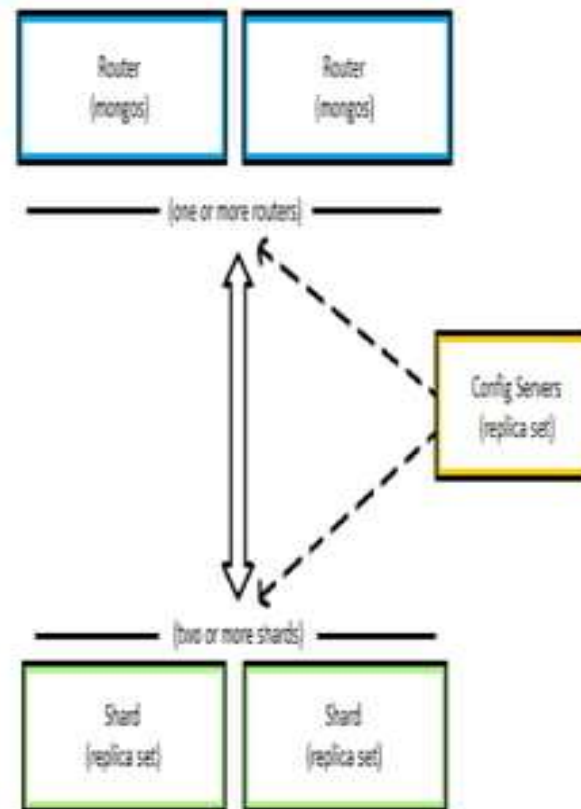
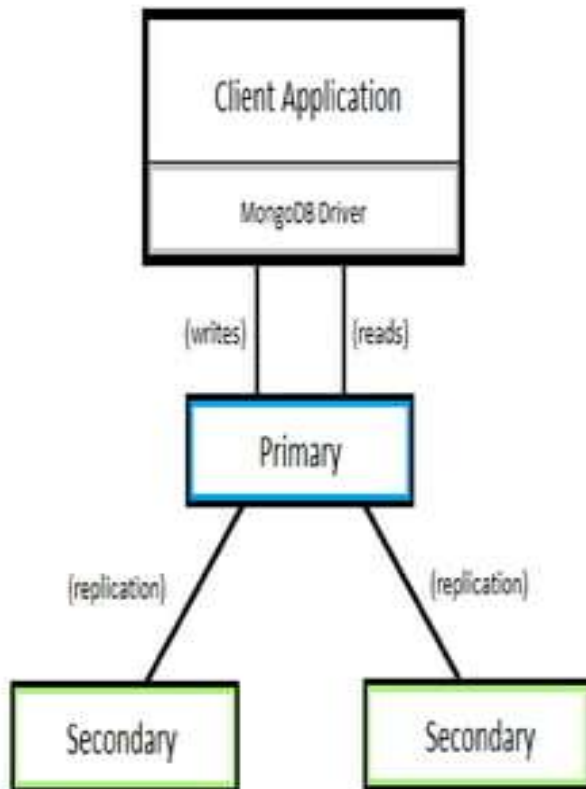
SHARDING

- Sharding is a technique of splitting up a large collection amongst multiple servers. When we shard, we deploy multiple mongod servers. And in the front, mongos which is a router. The application talks to this router. This router then talks to various servers, the mongods. The application and the mongos are usually co-located on the same server. We can have multiple mongos services running on the same machine. It's also recommended to keep set of multiple mongods (together called **replica set**), instead of one single mongod on each server. A replica set keeps the data in sync across several different instances so that if one of them goes down, we won't lose any data. Logically, each replica set can be seen as a **shard**. It's transparent to the application, the way MongoDB chooses to shard is we choose a **shard key**.



Replication ***VS*** ***Sharding***



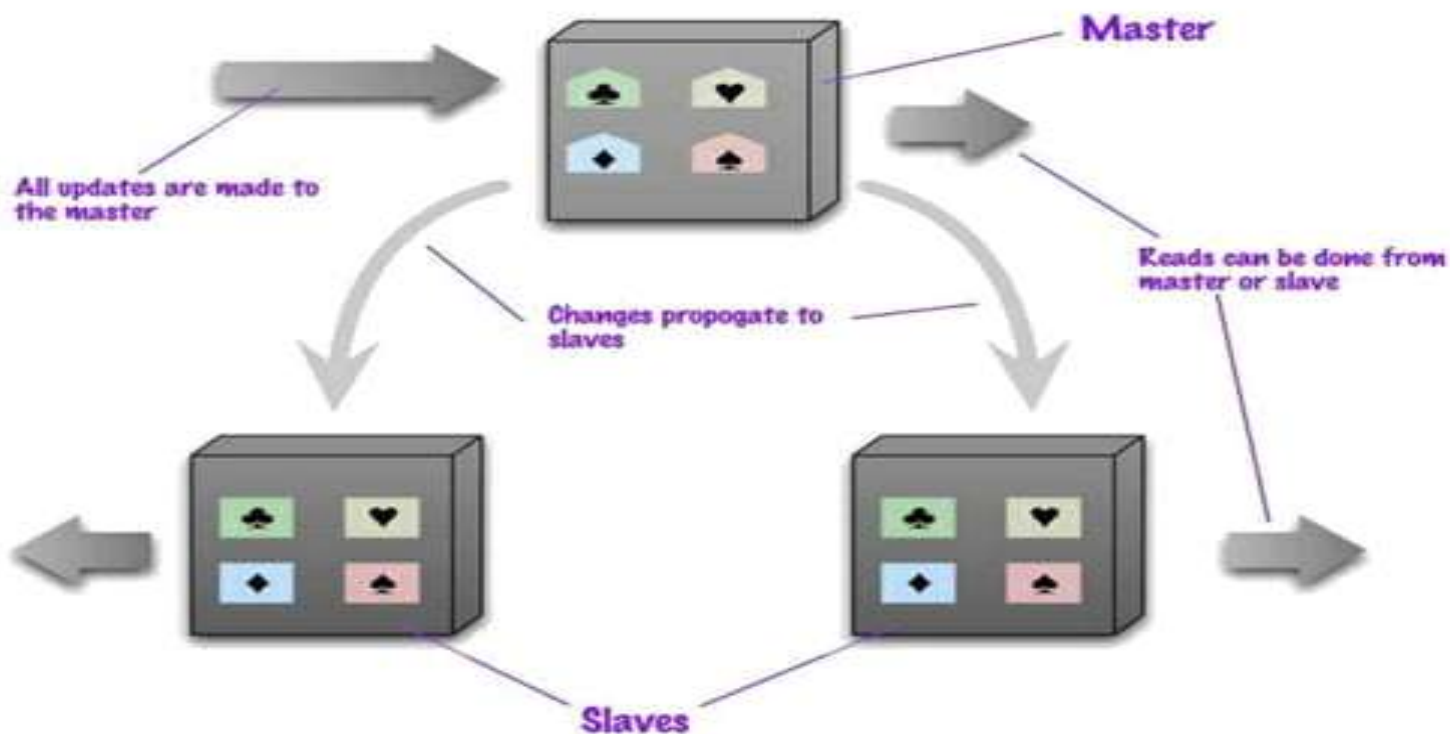




Master-Slave Replication

- With master-slave distribution, you **replicate data across multiple nodes**. One node is designated as the master, or primary. This master is the authoritative source for the data and is usually responsible for processing any updates to that data. The other nodes are **slaves, or secondaries**. A replication process synchronizes the slaves with the master

Master-Slave Replication





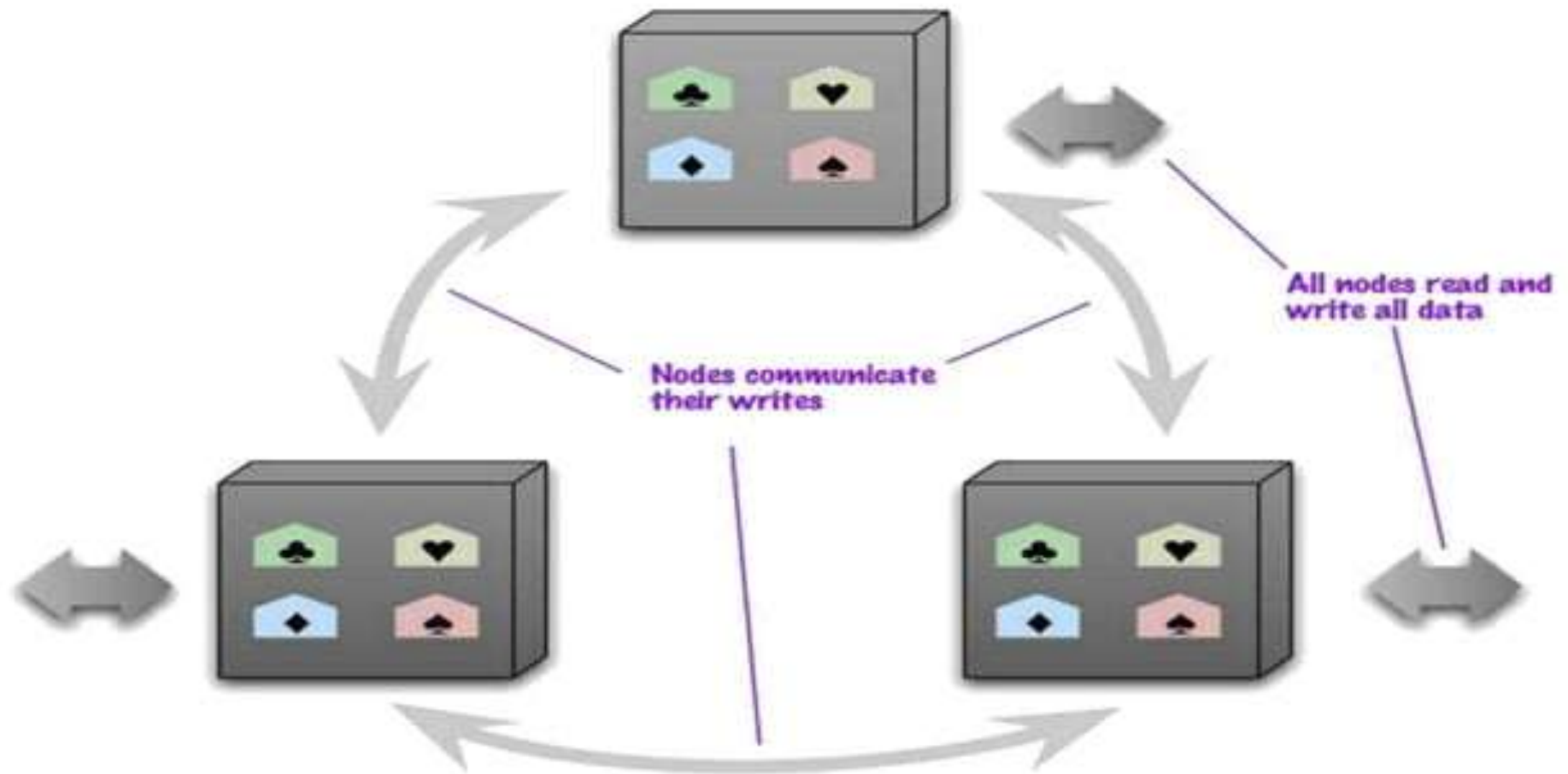
- advantage of master-slave replication is **read resilience**: Should the master fail, the slaves can still handle read requests. Again, this is useful if most of your data access is reads. The failure of the master does eliminate the ability to handle writes until either the master is restored or a new master is appointed. However, having slaves as replicates of the master does speed up recovery after a failure of the master since a slave can be appointed a new master very quickly.



Peer-to-Peer Replication

- Master-slave replication helps with read scalability but doesn't help with scalability of writes. It provides resilience against failure of a slave, but not of a master. Essentially, the master is still a bottleneck and a single point of failure. Peer-to-peer replication attacks these problems by not having a master. All the replicas have equal weight, they can all accept writes, and the loss of any of them doesn't prevent access to the data store.

**Peer-to-peer replication has all nodes
applying reads and writes to all the
data.**





consistency

- With a peer-to-peer replication cluster, you can ride over node failures without losing access to data.
- We can easily add nodes to improve your performance. There's much to like here—but there are complications.
- The biggest complication is, again, **consistency**. When you can write to two different places, you run the risk that two people will attempt **to update the same record at the same time—a write-write conflict**.
- Inconsistencies on read lead to problems but at least they are relatively transient.

In every lecture, Faculty may talk on any of the following:

- **DCS & GSFCU Portal:** Remind students to use DCS, talk about it's features and instruct all to visit GSFCU website
- **Discipline, Values and Ethics.**
- **GSFC University – My University:** Emphasize the importance of taking ownership of the University.
- **Respect & Courtesy:** Remind students to be respectful to peers, teachers, and staff. Encourage polite language and behavior.
- **Punctuality:** Emphasize the importance of arriving on time to class and completing assignments by deadlines.
- **Classroom Rules:** Reinforce basic classroom rules, such as keeping mobile phones on silent and maintaining a clean environment.
- **Safety:** Remind students about safety protocols, whether it's physical safety in the classroom or digital safety when using online resources.
- **Positive Attitude:** Encourage a positive mindset, cooperation and supporting one another.

GSFCU & Values

Respect & Courtesy



Dress
Appropriately



Respect
Personal Space



Be Mindful
of Your Words



Respect
Coworkers' Time



Keep Your
Workplace Clean



Socialise with
Your Peers



Use Your
Mobile Wisely



Take
Responsibility